

PRACTICAL- 9 (A)

AIM

Natural Language Processing

a. Data Cleaning using NLTK Toolkit.

THEORY

In Natural Language Processing (NLP), raw text data often contains unnecessary elements such as punctuation, special characters, stopwords, and inconsistent formatting. Data cleaning is the process of preparing and transforming this raw text into a structured and usable format for analysis.

The NLTK (Natural Language Toolkit) is a widely used Python library that provides tools and resources for working with human language data. It helps perform several preprocessing tasks required for text analysis and machine learning.

Data cleaning using NLTK typically involves the following steps:

- Tokenization: Splitting text into smaller units such as words or sentences.
- Lowercasing: Converting all characters to lowercase to maintain uniformity.
- Removing Stopwords: Eliminating commonly used words such as *the*, *is*, *and*, *in*, which usually do not contribute much meaning.
- Removing Punctuation and Special Characters: Cleaning symbols and punctuation marks that do not add semantic value.
- Stemming or Lemmatization: Reducing words to their root or base form to standardize similar words.

These preprocessing steps help improve the quality of text data and make it suitable for tasks such as text classification, sentiment analysis, and information retrieval. By using the NLTK toolkit, developers can efficiently clean and prepare textual data for further NLP processing.

SOFTWARE USED

Jupyter Notebook

CODE

```
import nltk  
  
import string  
  
from nltk.tokenize import word_tokenize
```

```

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download resources
nltk.download('punkt')
nltk.download('stopwords')

text = "Natural Language Processing is an exciting field of Artificial Intelligence!"

# Tokenization
tokens = word_tokenize(text)

# Remove punctuation
tokens = [word for word in tokens if word not in string.punctuation]

# Remove stopwords
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in tokens if word.lower() not in stop_words]

# Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_words]

print("Tokens:", tokens)
print("After Stopword Removal:", filtered_words)
print("After Stemming:", stemmed_words)

```

OUTPUT

```

[nltk_data] Downloading package punkt to /Users/mudit/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /Users/mudit/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
Tokens: ['Natural', 'Language', 'Processing', 'is', 'an', 'exciting', 'field', 'of', 'Artificial', 'Intelligence']
After Stopword Removal: ['Natural', 'Language', 'Processing', 'exciting', 'field', 'Artificial', 'Intelligence']
After Stemming: ['natur', 'languag', 'process', 'excit', 'field', 'artifici', 'intellig']

```

PRACTICAL- 9 (B)

AIM

Natural Language Processing

b. To implement Frequency-Based Word Embedding Techniques Bag of Words 2. TF-IDF

THEORY

In Natural Language Processing (NLP), text data must be converted into numerical form so that computers can process it. Frequency-based word embedding techniques represent text using the frequency or importance of words in a document.

Bag of Words (BoW)

The Bag of Words model converts text into numerical vectors by counting how many times each word appears in a document. It creates a vocabulary of all unique words and represents each document as a vector of word frequencies. This method ignores grammar and word order and focuses only on the presence or frequency of words.

TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF is an improved version of Bag of Words that measures how important a word is in a document compared to all documents in the dataset.

- Term Frequency (TF): Number of times a word appears in a document.
- Inverse Document Frequency (IDF): Measures how rare or important a word is across all documents.

The TF-IDF value is calculated as:

$$TF-IDF = TF \times IDF$$

Words that appear frequently in a document but rarely in other documents receive higher TF-IDF scores, making them more important.

These techniques are widely used in text classification, sentiment analysis, and information retrieval.

SOFTWARE USED

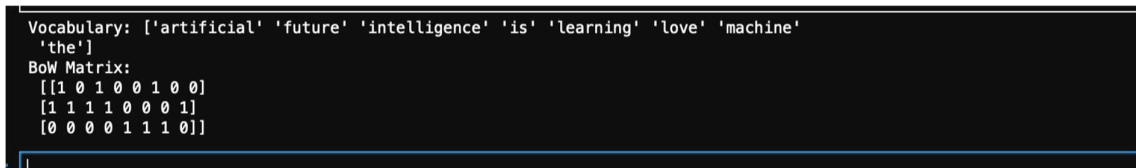
Jupyter Notebook

CODE

```
documents = [
```

```
"I love artificial intelligence",  
"Artificial intelligence is the future",  
"I love machine learning"  
]  
  
vectorizer = CountVectorizer()  
  
X = vectorizer.fit_transform(documents)  
  
print("Vocabulary:", vectorizer.get_feature_names_out())  
print("BoW Matrix:\n", X.toarray())
```

OUTPUT



```
Vocabulary: ['artificial' 'future' 'intelligence' 'is' 'learning' 'love' 'machine'  
            'the']  
BoW Matrix:  
[[1 0 1 0 0 1 0 0]  
 [1 1 1 0 0 0 1]  
 [0 0 0 1 1 1 0]]
```

CODE (TF-IDF)

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
documents = [  
    "I love artificial intelligence",  
    "Artificial intelligence is the future",  
    "I love machine learning"  
]  
  
vectorizer = TfidfVectorizer()  
  
X = vectorizer.fit_transform(documents)
```

```
print("Vocabulary:", vectorizer.get_feature_names_out())  
print("TF-IDF Matrix:\n", X.toarray())
```

OUTPUT

```
Vocabulary: ['artificial' 'future' 'intelligence' 'is' 'learning' 'love' 'machine'  
            'the']  
TF-IDF Matrix:  
[[0.57735027 0.          0.57735027 0.          0.          0.57735027  
  0.          0.          ]  
 [0.37302199 0.49047908 0.37302199 0.49047908 0.          0.  
  0.          0.49047908]  
 [0.          0.          0.          0.          0.62276601 0.4736296  
  0.62276601 0.          ]]
```